Software Requirements Specification

for

II-19 QueuePilot

Version 1.0

Prepared by

Group Number: II-19

Om Kumar 25CSB0A20
Abhinav Singh Saini 25CSB0A50
Samyak Jain 25CSB0A04
Subhamjyoti Maharana 25CSB0A05

Project Owner:

Course: CS1207 Object Oriented Programming Lab

Date:

Contents

1 Introduction....3
1.1 Document Purpose....3
1.2 Product Scope....3
1.3 Definitions, Acronyms and Abbreviations....4
1.4 Document Conventions....4
1.5 References and Acknowledgments....4
2 Overall Description....5
2.1 Product Overview....5
2.2 Product Functionality....6
2.3 Design and Implementation Constraints....6
2.4 Assumptions and Dependencies....6
3 Specific Requirements....7
3.1 Functional Requirements....7
3.3 Use Case Model....8
3.3.1 Use Case #1 - Submit and Manage Jobs (U1)....9
3.3.2 Use Case #2 - Manage Scheduler and Monitor Platform (U2)....10
3.3.3 Use Case #3 - Schedule and Dispatch Jobs (U3)....11
3.3.4 Use Case #4 - Execute Assigned Job (U4)....12
3.3.5 Use Case #5 - Detect Worker Failure and Recover Work (U5)....13
4 Other Non-functional Requirements....14
4.1 Performance Requirements....14
4.2 Safety and Security Requirements....14
5 Other Requirements....14

Revisions			
Version	**Primary Author(s)**	**Description of Version**	**Date Completed**
Draft 1.0	Om Kumar, Abhinav Singh Saini, Samyak Jain, Subhamjyoti Maharana	Initial SRS draft for QueuePilot.	

1 Introduction

QueuePilot is a job scheduling and worker orchestration platform proposed for the CS1207 Object Oriented Programming Lab. It accepts jobs through an API, organizes them into queues, selects runnable work using configurable policies, and dispatches tasks to registered workers. It also records job progress and outcomes, monitors worker health, and supports controlled recovery when a worker stops responding.

This section states the purpose of the requirements document and the scope of the software. Later sections describe the system environment, functions, detailed requirements, use cases, performance expectations, and security safeguards.

1.1 Document Purpose

This Software Requirements Specification (SRS) defines the requirements for QueuePilot Version 1.0. It establishes the intended behaviour and boundaries of the project so that all four team members can design, implement, test, and demonstrate the same system. It is intended for the student development team, the course teaching staff, and anyone reviewing the project.

The SRS covers job submission and tracking, scheduling policies, queue management, worker registration and execution, heartbeat and lease monitoring, retries and reassignment, a web dashboard, and scheduling benchmarks. It describes the initial implementation as one scheduler coordinator with concurrent worker threads. Separate worker processes are treated as a possible extension, not a required first-release capability.

1.2 Product Scope

QueuePilot aims to make task execution easier to manage and easier to observe. Clients will be able to submit supported jobs, obtain a job identifier, check status, and retrieve results. The scheduler will choose work using First Come, First Served (FCFS), Priority Scheduling with Aging, or Shortest Job First (SJF), then assign it only to a healthy worker that has the required capability.

The platform will track job state, attempts, timestamps, progress, results, and errors. If a worker becomes unresponsive, the system will detect an expired heartbeat or lease and retry or reassign eligible work within a configured retry limit. Administrators will use the dashboard to manage worker metadata and scheduling settings, inspect system health, simulate worker failures, and compare scheduling policies using measurable workload results.

The intended benefit is a demonstrable, modular OOP application combining algorithms, concurrency, persistence, fault handling, and monitoring. QueuePilot is an educational prototype; the first release does not promise production-grade distributed consensus, exactly-once execution, or support for arbitrary user-supplied executable code.

1.3 Definitions, Acronyms and Abbreviations

API - Application Programming Interface. The interface clients use to send requests to QueuePilot.

DAG - Directed Acyclic Graph. A dependency graph with no cycles; prerequisite jobs must complete before dependent jobs become runnable.

FCFS - First Come, First Served. A scheduling policy that selects the earliest eligible job first.

Heartbeat. A periodic signal sent by a worker to indicate that it is still responsive.

Job. A unit of work submitted to QueuePilot for validation, scheduling, execution, and tracking.

Lease. A time-limited assignment record that lets the scheduler identify abandoned or expired work.

Priority Aging. A policy adjustment that increases a job's effective priority while it waits.

SJF - Shortest Job First. A scheduling policy that prefers eligible jobs with shorter estimated execution times.

SRS - Software Requirements Specification. The document defining the software requirements.

Worker. An execution unit that performs an approved job type and reports progress or results.

1.4 Document Conventions

This document follows the section order and presentation guidance in the course-provided SRS template. Body text uses Arial 11-point type, single spacing, and one-inch page margins. Section and subsection headings are bold and numbered. Functional requirements are labelled F1, F2, and so on; use cases are labelled U1, U2, and so on. The word “shall” identifies a required system behaviour. Comments or explanatory notes are italicized where used. Diagrams are presented in black and white.

1.5 References and Acknowledgments

- CS1207 Object Oriented Programming Lab, course-provided Software Requirements Specification template and sample SRS supplied to the team.
- The project name, scope, and feature descriptions in this SRS are prepared by the QueuePilot team for the course project.

2 Overall Description

2.1 Product Overview

QueuePilot is a new, self-contained software prototype. Its main external participants are clients, who submit and inspect jobs; administrators, who manage scheduling settings and observe the system; and worker agents, which execute assigned jobs and send heartbeats, progress updates, and results. The web dashboard calls the REST API, while the scheduler, job management service, worker registry, recovery manager, and database coordinate the internal workflow.

The diagram below shows the major components and their information flow. For the initial implementation, the scheduler and worker threads run within one application deployment. The design separates responsibilities so that worker processes could be explored later without making multi-machine operation a release requirement.

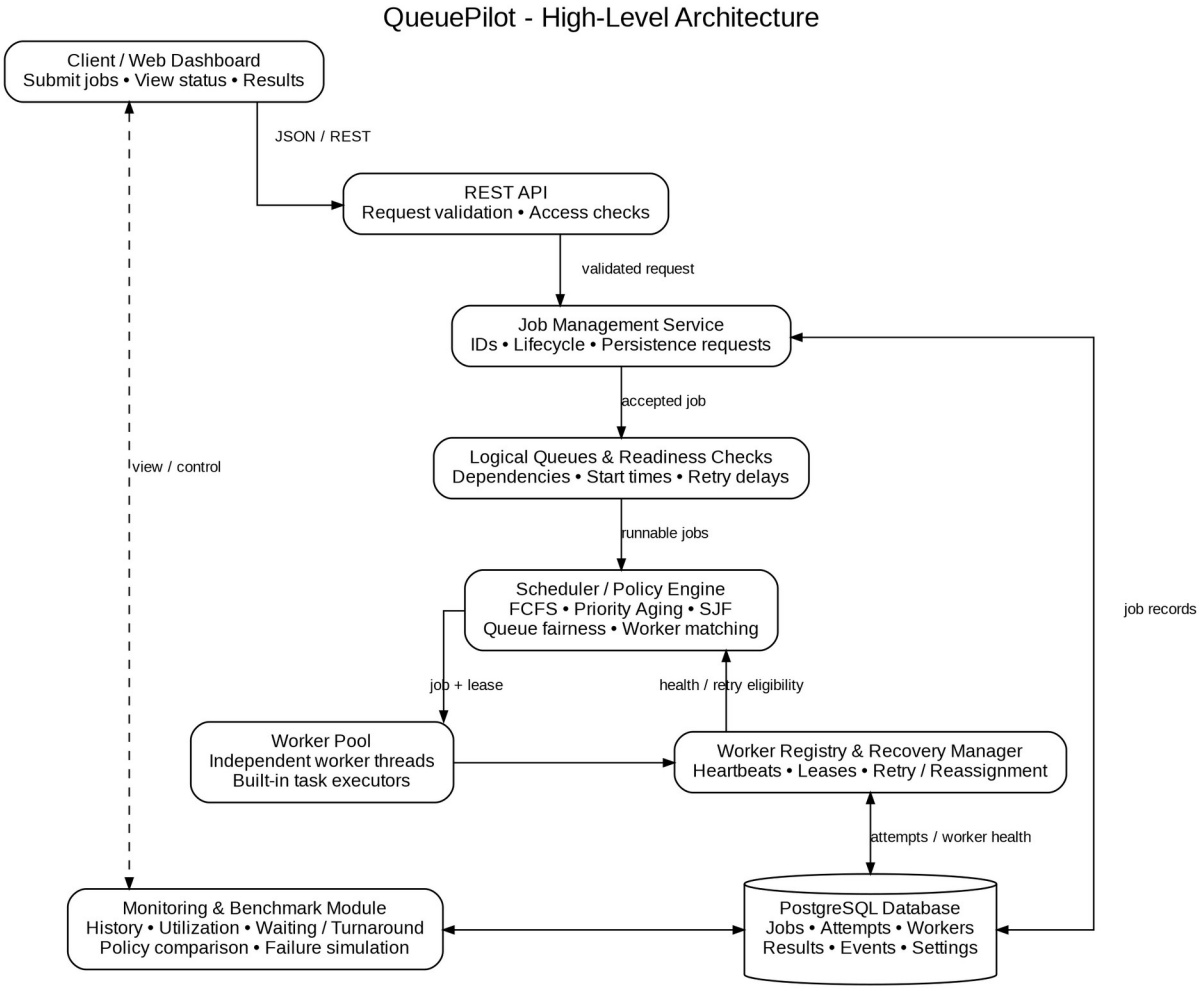


Figure 1. QueuePilot high-level architecture

2.2 Product Functionality

- Accept and validate job submissions through a REST API.
- Persist job records, execution attempts, results, status changes, and worker metadata.
- Maintain logical queues and select runnable jobs using configurable scheduling policies.
- Register workers, record their capabilities, and dispatch jobs only to eligible workers.
- Execute supported built-in tasks concurrently and report progress, completion, and errors.
- Monitor heartbeats and leases, detect worker failures, and perform bounded retries or reassignment.
- Support job priorities, aging, per-queue fairness, scheduled starts, deadline warnings, and dependencies.
- Provide a dashboard for monitoring jobs and workers, inspecting history, simulating failures, and comparing scheduling metrics.

2.3 Design and Implementation Constraints

The planned implementation uses Java, Spring Boot for REST services, PostgreSQL for persistent storage, and a browser-based interface built with HTML, CSS, and JavaScript. Maven will manage the Java build; JUnit will be used for automated tests; and Git will be used for version control. Exact library versions may be selected during setup.

The first release is designed for a single coordinator and worker threads on a development machine. The project timeline is approximately four to six weeks, so the team will prioritize the core job lifecycle, scheduling, and recovery flow before optional extensions. Only predefined, validated job types will execute; arbitrary shell commands or client-supplied executable code are outside scope.

2.4 Assumptions and Dependencies

- A compatible Java Development Kit, Maven, PostgreSQL instance, and modern web browser will be available during development and demonstration.
- Worker capabilities and estimated job durations are declared or configured; QueuePilot does not require an ML model to predict runtime.
- Heartbeat intervals, lease expiry, retry count, and retry delay are configurable test parameters.
- For the prototype, built-in task types are designed to be safe to retry; external side effects and exactly-once execution are not guaranteed.
- The dashboard, API, scheduler, workers, and database are expected to run in a controlled development environment with network access limited to the application.

3 Specific Requirements

3.1 Functional Requirements

The following requirements define the observable functions of QueuePilot. F1-F15 form the baseline specification for the first release; any extension that is deferred should be recorded by the team before implementation.

F1: The system shall expose a REST API through which a client can submit a job containing a supported job type and its required input data.

F2: The system shall validate each submission, reject missing or malformed fields with a useful error response, and assign a unique identifier to every accepted job.

F3: The system shall persist job metadata, status, timestamps, attempt history, output, and error details; clients shall be able to query status and results and cancel a job before execution starts.

F4: The system shall support FCFS, Priority Scheduling with Aging, and SJF policies, with the active policy selected through an administrator control.

F5: The scheduler shall select only runnable jobs, considering current state, scheduled start time, dependency completion, retry delay, and applicable queue rules.

F6: The system shall maintain a worker registry containing worker identity, supported capabilities, health status, concurrency limit, and current assignments.

F7: The scheduler shall assign a job only to a healthy and available worker whose declared capabilities satisfy the job requirements; otherwise the job shall remain queued.

F8: The system shall provide a defined set of built-in job executors, including sorting, text analysis, and mathematical computation; new executor types shall follow a common interface.

F9: Workers shall report progress, completion, output, and errors; QueuePilot shall record these events and update the job lifecycle accordingly.

F10: Each active worker assignment shall use a time-limited lease or attempt token, and workers shall send periodic heartbeats while active.

F11: When a worker or lease is judged unresponsive, the system shall mark the worker unavailable and retry or reassign eligible abandoned work up to a configured limit; stale updates from an expired attempt shall not overwrite a newer attempt.

F12: The system shall allow a job to depend on prerequisite jobs, reject dependency cycles, and keep a dependent job blocked until all prerequisites have succeeded.

F13: The system shall support future scheduled start times and target deadlines; the dashboard shall indicate jobs that are approaching or have passed their deadlines.

F14: The system shall provide basic queue-level fairness, such as round-robin selection across eligible queues, and configurable per-client submission limits to reduce monopolization.

F15: The dashboard shall display job and worker health, execution history, and scheduling metrics; it shall support controlled worker-failure simulation and benchmark comparisons for waiting time, turnaround time, throughput, and worker utilization.

3.3 Use Case Model

The model identifies the primary interactions between the QueuePilot system and its actors. Client actions focus on submitting and managing jobs; administrator actions focus on configuration and monitoring; worker-agent actions report execution and health. Detailed specifications for the principal workflows follow.

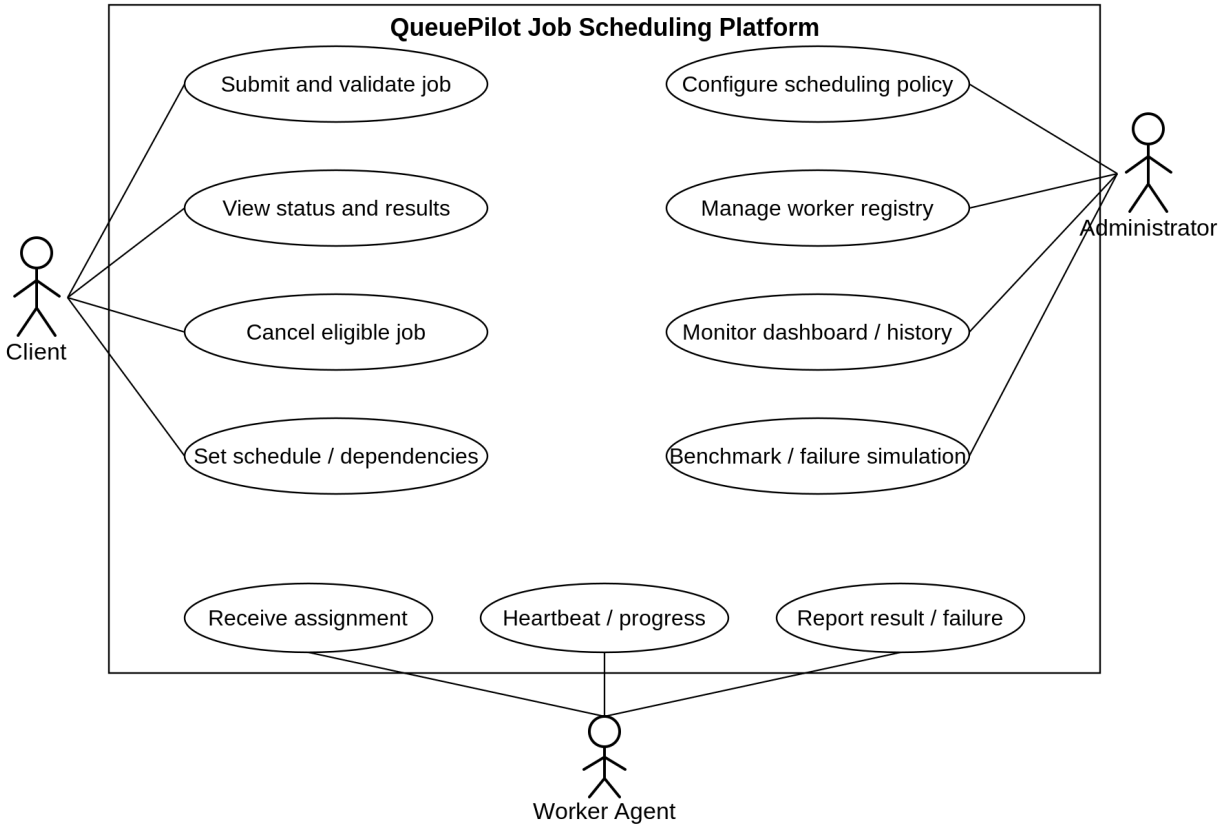


Figure 2. QueuePilot use-case model

3.3.1 Use Case #1 - Submit and Manage Jobs (U1)

Author - Abhinav Singh Saini (25CSB0A50)

Purpose - Allow a client to submit a supported job and subsequently inspect or cancel it when cancellation is still permitted.

Requirements Traceability - F1, F2, F3, F5, F12, F13

Priority - High

Preconditions - The QueuePilot API is available; the client can reach the submission endpoint; the requested job type is supported.

Postconditions - An accepted job is stored with a unique ID and a state such as QUEUED, SCHEDULED, or BLOCKED. An invalid request is rejected without creating a runnable job.

Actors - Client (primary); QueuePilot API and job-management service (supporting components)

Extends - N/A

Includes - Validate Job Input; Persist Job Record

Flow of Events

1. Basic Flow

1. The client submits the job type, required input, priority, and any optional capability, schedule, deadline, or dependency fields.
2. The API validates the request structure and job-specific input, then checks that referenced prerequisites exist and do not create a dependency cycle.
3. The system generates a unique job ID and stores the accepted job, metadata, and initial event in the database.
4. The system sets the initial state to SCHEDULED when the start time is in the future, BLOCKED when prerequisites are incomplete, or QUEUED when it is otherwise eligible for scheduling.
5. The API returns the job ID and current state. The client may later query status and result details, inspect recorded history, or cancel the job if it has not started.

2. Alternative Flow

- If the requested start time is in the future, the job remains SCHEDULED until that time.
- If prerequisites have not completed, the job remains BLOCKED until all required jobs succeed.
- If a cancellable job is cancelled before execution begins, the system records CANCELLED and the cancellation event.

3. Exceptions

- Missing fields, unsupported job type, invalid payload, unknown prerequisite, or dependency cycle: return a validation error and do not enqueue the job.
- Database or API error: return a failure response; do not report a job as accepted unless its record was persisted.

Notes/Issues - Job execution is asynchronous; successful submission confirms acceptance, not completion.

3.3.2 Use Case #2 - Manage Scheduler and Monitor Platform (U2)

Author - Om Kumar (25CSB0A20)

Purpose - Allow an administrator to configure scheduling behaviour, manage worker metadata, inspect system health, and run diagnostic benchmarks.

Requirements Traceability - F4, F6, F14, F15

Priority - High

Preconditions - The application is running and the administrator has access to the management controls.

Postconditions - Valid settings are saved and applied to subsequent scheduling decisions; worker and system information is available in the dashboard; benchmark results are recorded.

Actors - Administrator

Extends - N/A

Includes - Validate Settings; Update Worker Registry; Calculate Scheduling Metrics

Flow of Events

1. Basic Flow

1. The administrator opens the dashboard and selects a scheduling policy: FCFS, Priority with Aging, or SJF.
2. The system validates policy-specific settings, saves the choice, and displays the currently active policy.
3. The administrator may register or update worker capabilities and concurrency limits, or mark a worker unavailable for new assignments.
4. The dashboard displays job counts by state, worker health, recent failures, queue information, and execution history.
5. The administrator may start a controlled benchmark using a repeatable workload or simulate a worker failure; QueuePilot records the test configuration and calculates the available metrics.

2. Alternative Flow

- If a worker has an active assignment, the system may stop it receiving new jobs while allowing its current job to finish, unless a failure simulation is explicitly requested.
- A benchmark may be run in simulation mode so each scheduling policy can be compared on the same workload.

3. Exceptions

- Invalid policy or worker settings: the system rejects the change and retains the last valid configuration.
- A benchmark cannot start while its required workload is invalid; the dashboard displays an explanation.

Notes/Issues - Administrative controls should be limited to an authorized administrator session or configured administrative credential.

3.3.3 Use Case #3 - Schedule and Dispatch Jobs (U3)

Author - Samyak Jain (25CSB0A04)

Purpose - Select the next runnable job according to the configured policy and dispatch it to a healthy worker that can execute it.

Requirements Traceability - F4, F5, F6, F7, F14

Priority - High

Preconditions - At least one job is present in a logical queue; a valid policy is configured; worker status and capability information are available.

Postconditions - A selected job is recorded as assigned or running with an attempt identifier and lease; or it remains queued/blocked if no valid assignment can be made.

Actors - Worker Agent (available capacity signal); QueuePilot scheduling service (internal trigger)

Extends - N/A

Includes - Check Job Readiness; Select Scheduling Policy; Match Worker Capability

Flow of Events

1. Basic Flow

1. The scheduler gathers jobs that are eligible by state, scheduled start time, dependency status, and retry delay.
2. The scheduler applies the active policy: FCFS ordering, effective priority with aging, or shortest estimated duration. Queue fairness rules are applied where configured.
3. The scheduler filters registered workers by health, available capacity, and required capability tags.
4. The scheduler selects a matching worker, creates an attempt record and time-limited lease, and dispatches the job to that worker.
5. The job and worker states are updated and the assignment is recorded in persistent history.

2. Alternative Flow

- If there is no healthy worker with the required capability, the job remains queued and can be reconsidered when worker availability changes.
- If the job is not yet ready because of a dependency, scheduled start, or retry delay, it is not dispatched in this cycle.

3. Exceptions

- If the worker becomes unavailable before accepting the assignment, the assignment is released or allowed to expire and the job is reconsidered.
- If persistence of the assignment fails, the system must not leave the job marked as successfully dispatched.

Notes/Issues - The scheduler uses estimated execution duration for SJF; measured runtime may differ from the estimate.

3.3.4 Use Case #4 - Execute Assigned Job (U4)

Author - Subhamjyoti Maharana (25CSB0A05)

Purpose - Execute an assigned built-in task and report progress, output, completion, or failure to QueuePilot.

Requirements Traceability - F8, F9, F10

Priority - High

Preconditions - A worker has received an assignment with a valid job ID, attempt identifier, and lease; a matching built-in executor exists.

Postconditions - The job is marked COMPLETED with a stored result, or it is marked for retry/FAILED with an error and attempt history.

Actors - Worker Agent

Extends - N/A

Includes - Validate Assignment Lease; Execute Built-in Task; Persist Result

Flow of Events

1. Basic Flow

1. The worker accepts the assignment and verifies that its attempt identifier and lease are still valid.
2. The worker selects the built-in executor matching the job type and begins execution.
3. While running, the worker sends heartbeats and progress updates at configured intervals; the scheduler records the latest known state.
4. On success, the worker sends the result and completion status. QueuePilot stores the output and marks the job COMPLETED.
5. The system records timestamps and makes the result and execution history available through the API and dashboard.

2. Alternative Flow

- If execution fails with a retryable error, the worker reports the error and the recovery manager decides whether the job should be retried.
- If the job is cancelled before it begins, the worker does not execute it; cancellation of an already-running job is outside the guaranteed behaviour of the initial release.

3. Exceptions

- If the lease is expired or the assignment token is stale, the worker's completion update is rejected so it cannot overwrite a newer attempt.
- If a task executor raises an error, the error is logged and the attempt is reported as failed for retry evaluation.

Notes/Issues - Initial built-in tasks are selected to be safe to repeat. This reduces duplicate side effects but does not provide a general exactly-once guarantee.

3.3.5 Use Case #5 - Detect Worker Failure and Recover Work (U5)

Author - Abhinav Singh Saini (25CSB0A50)

Purpose - Detect an unresponsive worker and safely make eligible abandoned jobs available for another execution attempt.

Requirements Traceability - F9, F10, F11, F15

Priority - High

Preconditions - Workers have registered and are expected to send heartbeats; one or more assignments have active leases; recovery monitoring is enabled.

Postconditions - An unresponsive worker is marked unavailable. Each affected job is requeued for another attempt or marked FAILED if the retry limit has been reached; old attempt updates cannot replace newer state.

Actors - Worker Agent; Administrator (when initiating a failure simulation)

Extends - N/A

Includes - Check Heartbeat / Lease; Record Failure Event; Evaluate Retry Policy; Requeue Job

Flow of Events

1. Basic Flow

1. The recovery manager compares the last heartbeat and active lease with the configured timeout values.
2. When the timeout is exceeded, the system marks the worker unavailable and identifies its unfinished assignments.
3. For each affected job, the system marks the attempt abandoned and checks whether retry policy permits another attempt.
4. If retry is permitted, the job is placed back into the appropriate queue after any configured retry delay and can be assigned to another eligible worker.
5. If the attempt limit is exhausted, the job is marked FAILED with an explanatory error and complete attempt history.
6. The dashboard updates the worker state and records the recovery event. The same flow may be demonstrated using the failure-simulation control.

2. Alternative Flow

- If the worker resumes after being marked unavailable, it must re-register or become healthy again before receiving new assignments.
- If no suitable worker is available, a retryable job remains queued until a matching worker becomes available.

3. Exceptions

- If the database is unavailable during recovery, the system reports the problem and avoids claiming that reassignment was completed until the state is persisted.
- Late result or heartbeat messages from an expired attempt are ignored or rejected when a newer attempt is active.

Notes/Issues - Leases and attempt tokens reduce stale-update risks. Since a worker may fail after performing work, the platform is designed around bounded, retry-safe execution rather than exactly-once processing.

4 Other Non-functional Requirements

4.1 Performance Requirements

- In the reference development environment, a valid job submission or validation error should receive an API response within 2 seconds for requests of up to 64 KB, excluding job execution time.
- With 100 stored jobs and up to four worker threads in a local demonstration, dashboard summaries should reflect persisted status changes within 2 seconds under normal conditions.
- Worker failure detection shall occur after the configured heartbeat/lease threshold plus no more than one normal recovery polling interval.
- The benchmark module shall calculate average waiting time, average turnaround time, throughput, and worker utilization from recorded timestamps and counts; repeated runs with the same fixed workload and parameters should produce comparable results.

4.2 Safety and Security Requirements

- The system shall accept only supported, validated job types and shall not execute arbitrary shell commands or executable code supplied by a client.
- Input lengths, numeric ranges, dependency references, and configuration values shall be validated before they affect scheduling or execution.
- Worker-management, policy configuration, and failure-simulation controls shall be restricted to an authorized administrator session or configured administrative credential.
- Database credentials and administrative secrets shall not be committed to public source control or written into routine application logs.
- Expired assignment tokens and stale worker updates shall not overwrite a newer attempt. Retry limits shall prevent unbounded automatic retries.
- Error messages returned to clients shall describe the failure without exposing internal stack traces, credentials, or unrelated job data.

5 Other Requirements

- Data management: retain job input metadata, final output, status transitions, attempt records, worker health events, and benchmark results for review during the project demonstration.
- Maintainability: use separate modules and interfaces for job management, scheduling policies, task executors, worker monitoring, persistence, and dashboard services.
- Verification: test job validation, all scheduling policies, priority aging, capability matching, queue fairness, dependency-cycle rejection, lease expiry, retry limits, and stale-result rejection.
- Scope boundary: multi-node scheduler coordination, leader election, production-grade distributed consensus, arbitrary user code, and a strict exactly-once execution guarantee are not required for the initial release.